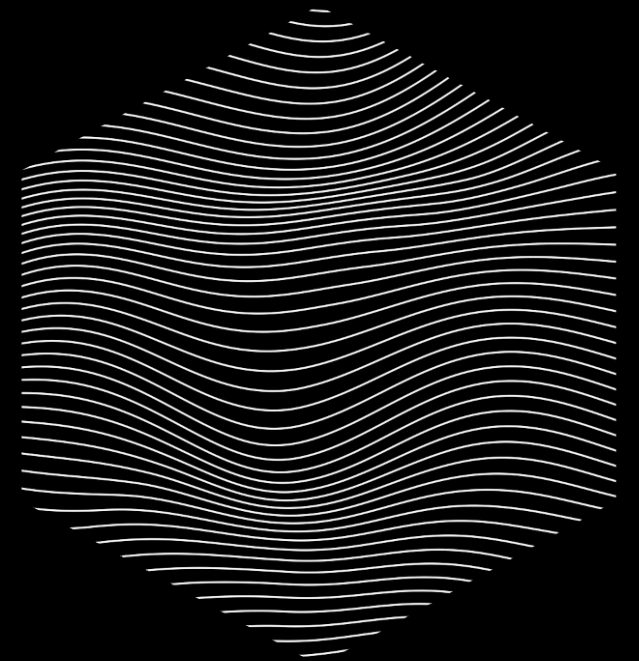




wingfoil

**the ultra low latency data streaming
framework**

August 2026



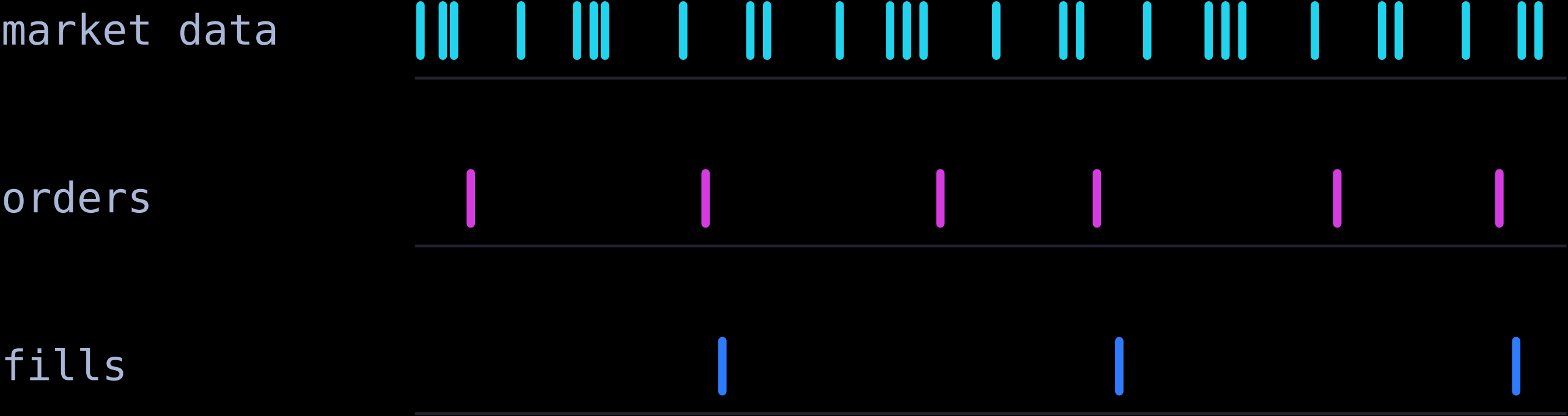
Jake Mitchell



25 years in financial services · JPMorgan · B2C2

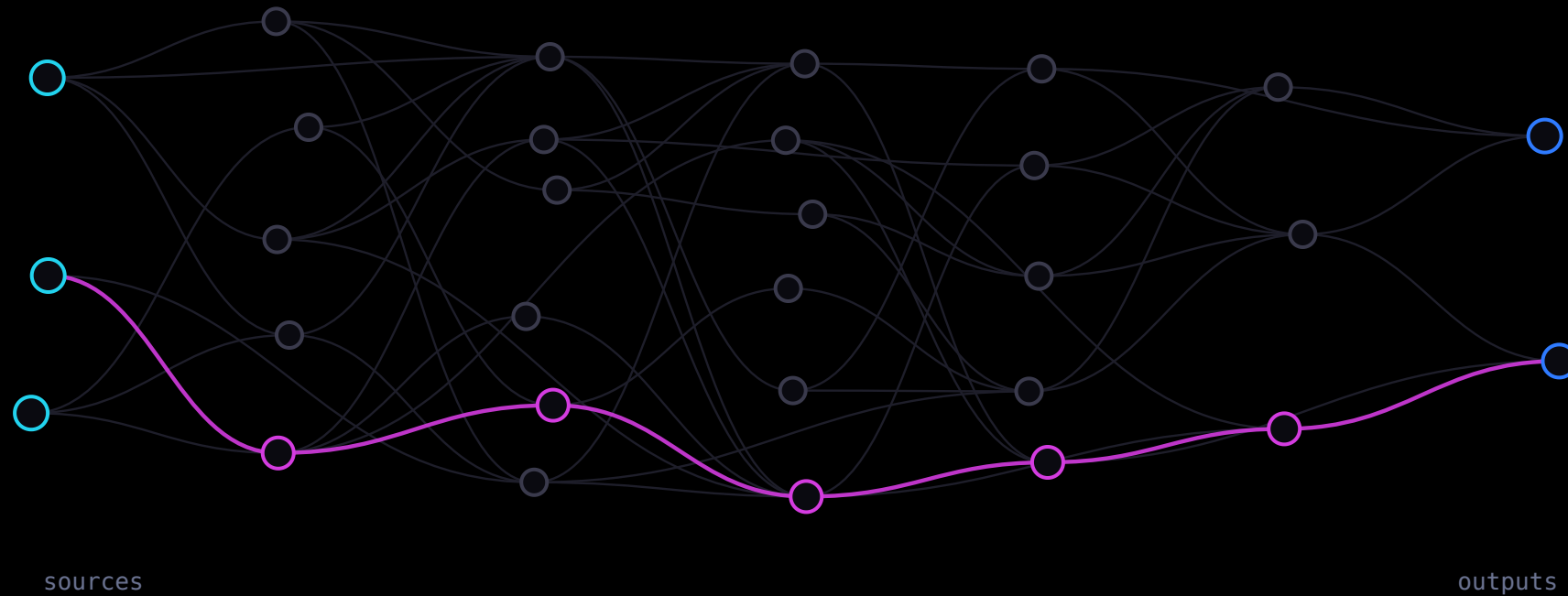
THE PROBLEM

Everything ticks at a different rate



THE SOLUTION

Wire a graph of calculations



- The framework abstracts over **what needs to tick, and when**.
- **Split and recombine** freely — it stays one graph.
- **Modulate the frequency** — window, throttle, sample.
- And it has to be **efficient**.

Wire it, build it, run it

```
let g = GraphBuilder::new();

let printed = g
    .ticker(Duration::from_millis(100))
    .count()
    .map(|i| format!("tick {i}"))
    .for_each(|msg: &String| { println!("{}", msg); Ok(()) });

g.build().run(RunMode::HistoricalFrom(NanoTime::ZERO), RunFor::Cycles(5));
```

```
historical run (instant):
```

```
tick 1
tick 2
tick 3
tick 4
tick 5
```

```
cargo run -p wingfoil --example hello_graph
```

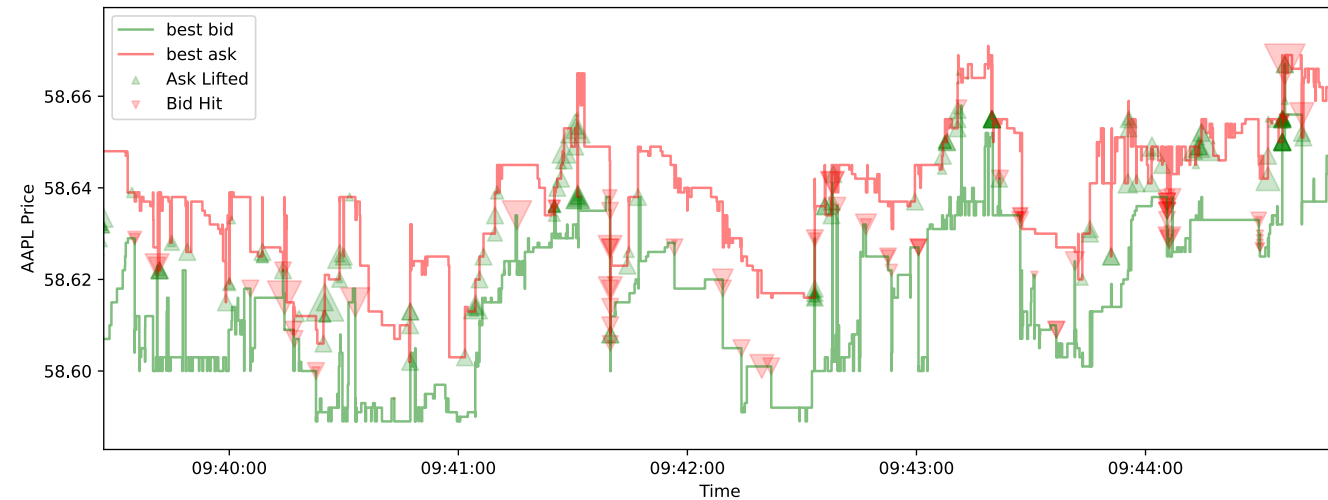
MOTIVATION, NOT THESIS

A real graph: NASDAQ limit orders → book → prices + fills

```
let (fills, prices) =  
  csv_read(&g, &src, get_time, true, None)?  
    .map(move |chunk: &Burst<Message>| {  
      process_orders(chunk, &book)  
    })  
    .split();  
  
prices.filter_none().distinct()  
  .csv_write(&prices_path)?;  
fills.csv_write(&fills_path)?;
```

Three different frequencies in one graph: many messages share a timestamp, top-of-book changes less often, trades are sparser still.

```
15040 two-way prices -> prices.csv  
4169 fills -> fills.csv  
in 119.704ms
```



I/O

Sixteen adapters, three patterns

MESSAGING

Kafka · ZeroMQ · Fluvio · Redis · etcd

LOW-LATENCY TRANSPORT

Aeron · iceoryx2

TRADING

FIX

STORAGE & FILES

kdb+ · PostgreSQL · CSV · lines

WEB

WebSocket client · WebSocket server

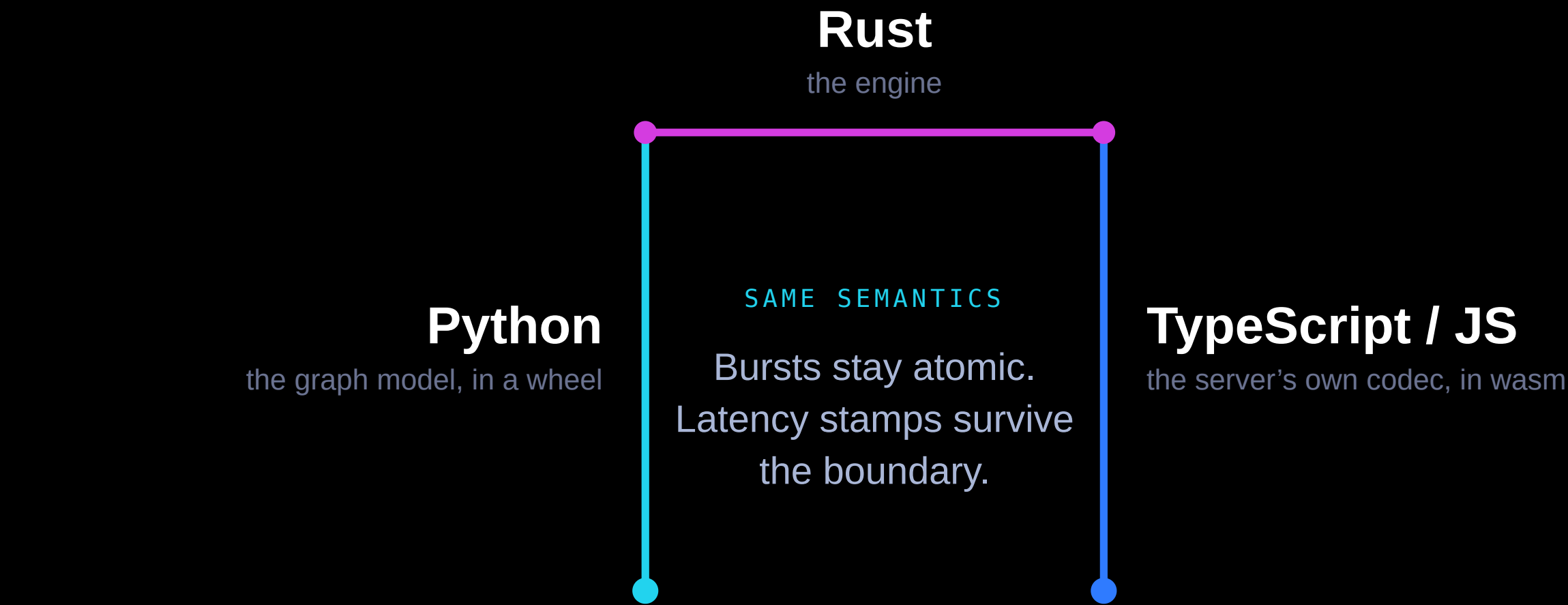
TELEMETRY

Prometheus · OpenTelemetry

Pattern	Latency	Run mode
Busy spin <i>Aeron · iceoryx2 · FIX</i>	Lowest — the node drains the port inside the cycle. <i>costs you a core</i>	Realtime
Worker thread <i>Kafka · ZeroMQ · etcd</i>	One channel hop. <i>thread parks; you keep the core</i>	Realtime
Async <i>kdb+ · PostgreSQL</i>	A hop plus an executor wake. <i>tokio, owned by the graph</i>	Realtime and historical

The core stays synchronous either way — **you are not locked into `async`**. iceoryx2 offers all three; you pick at wiring time.

Three languages, one engine



RUN MODES

Backtest it, then run it live

```
runner.run(RunMode::HistoricalFrom(NanoTime::ZERO), RunFor::Forever)?; // backtest  
runner.run(RunMode::RealTime, RunFor::Forever)?; // live
```

Two clocks, one of them yours

`time()` is source-driven — under replay it reads *no clock at all*. That is what makes a backtest deterministic.

A hang turned into an error

A live source rejects `HistoricalFrom` **at wiring time**, not at 3am.

PART TWO

Nitro

Performance, the wall we hit chasing it,
and the pattern that got us through.

THE HOOK

127 levels deep. 2^{127} distinct paths. One tick.

```
let mut source = g.constant(1_u128);  
for _ in 1..128 {  
    source = source.join(&source, |a, b| a + b);    // branch and recombine  
}
```

```
1 ticks processed in 12.953µs  
value 170141183460469231731687303715884105728
```

That value is 2^{127} — the correct answer, which is also exactly the number of source → sink paths a framework that propagates *one path at a time* would have to walk.

WHY IT STAYS FLAT

Visit each node once per tick

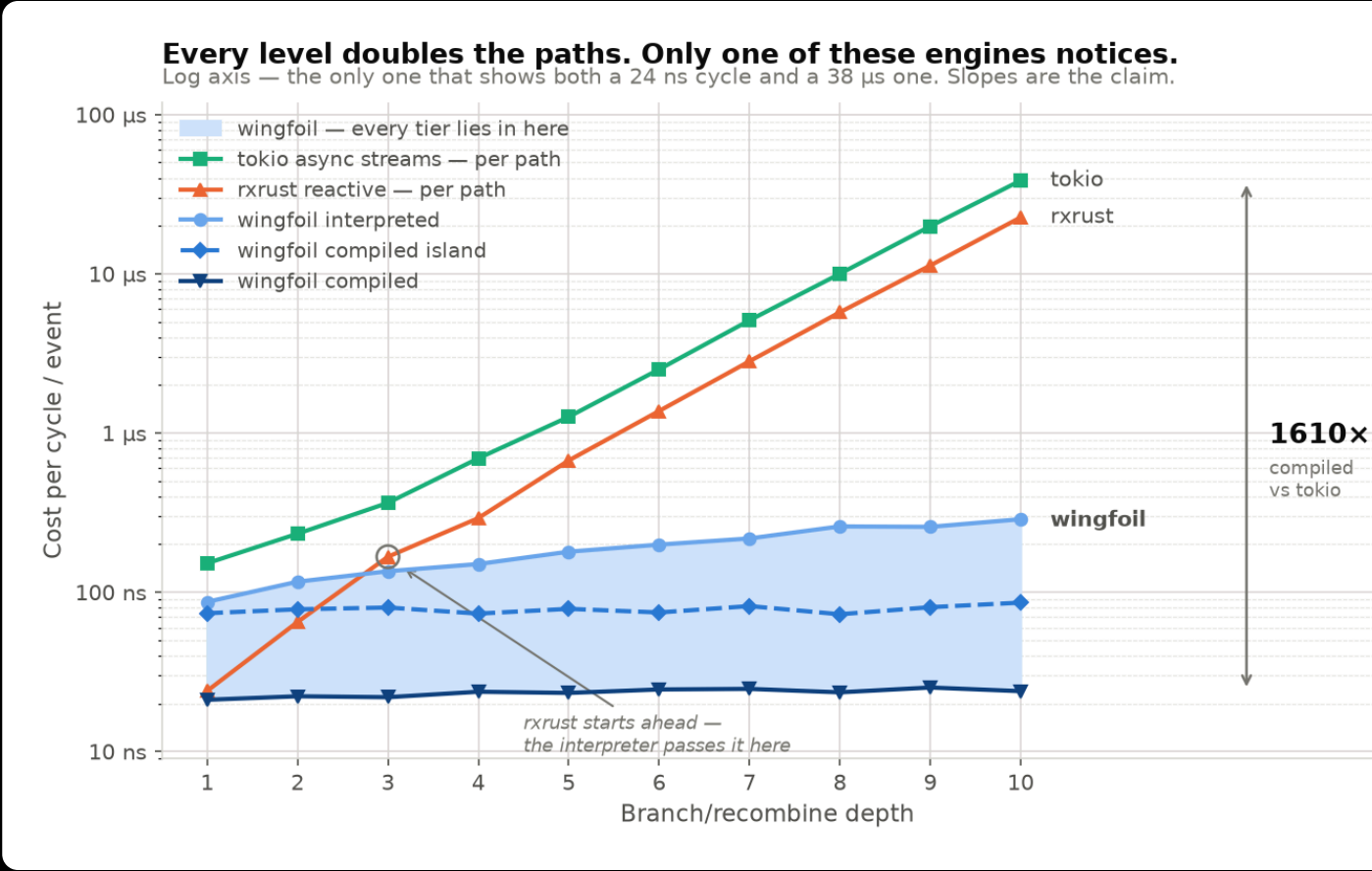
A node runs **after everything it reads**, exactly once — however many paths lead to it.

Propagate one path at a time and a shared node is re-visited per path. Branch and recombine, and that **doubles every level**.

Rx · async streams

Measured slopes 2.01× and 1.94× per level.

Wingfoil: flat.



THE ENGINEERING PROBLEM

What we wanted: compile the graph

Interpreted, every node pays a **dynamic dispatch** and a **trip through a value slot** — per node, per cycle.

On a hot chain that dominates the arithmetic you came to do.

So: **generate code**. One straight-line function, state in locals, optimiser working across node boundaries.

We built it

Ran the wiring from `build.rs`, walked the graph, emitted a runner.

It failed three ways — and the third told us why.

WHY A REWRITE, NOT A FEATURE

The generator failed three ways

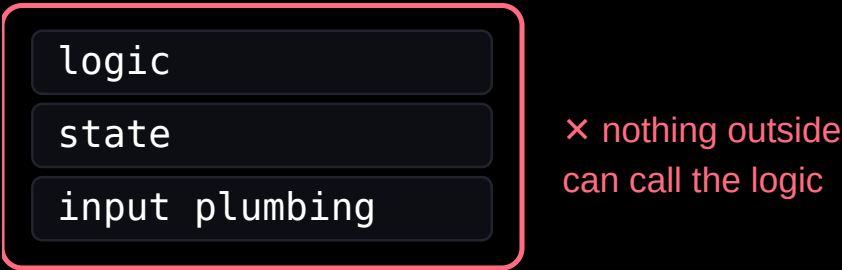
- 1 **It had to re-implement every node.** Semantics were welded to `RefCell` fields, so generated code could not *call* a node's cycle. Two `map` s, two `filter` s — nothing keeping them in step.
- 2 **Types and closures were already gone.** Wiring erased them. You cannot get `|x| x * 10` back out of a `Box<dyn Fn>` .
- 3 **Scheduling was inferred from names.** Nothing declared "I schedule myself", so the engine drove nodes off allowlists.

Node semantics were not *separately callable*.

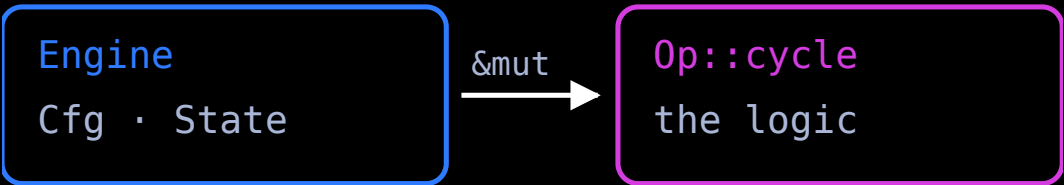
THE PATTERN

Inside-out

ONE OBJECT OWNS EVERYTHING



INSIDE-OUT



The tension

Encapsulation says bundle state with behaviour. Single responsibility says one reason to change. Follow the first far enough and you break the second.

The resolution

Encapsulate the **logic and the types**. Let **ownership** live outside.

Ownership moved out. Encapsulation didn't.
The op owns *what* and *how*. The engine owns *where* and *when*.

THE ONE DECISION EVERYTHING FOLLOWS FROM

Semantics became an associated function

```
trait Op {  
    type Cfg;           // construction-time config; closures live here  
    type State;         // engine-owned mutable state  
    type In<'a>;        // typed inputs, passed in per cycle  
    type Out;           // the produced value  
    const ACTIVATION: Activation;  
  
    fn cycle(cfg: &mut Self::Cfg, state: &mut Self::State,  
            input: Self::In<'_>, ctx: &mut Ctx<'_>) -> Result<Tick<Self::Out>>;  
}
```

Read what is *absent*. Nothing here says how state is stored, where inputs come from, or who calls it.

THE SAME NODE, BOTH WAYS

The node changed completely. The wiring barely moved.

legacy — the node owns everything

```
#[node(active = [upstream], output = value: f64)]
impl MutableNode for ScaleStream {
    fn cycle(&mut self, _state: &mut GraphState)
        -> Result<bool>
    {
        self.value =
            self.upstream.peek_value() * self.factor;
        Ok(true)
    }
}
```

owns: its upstream handles, its state, its output slot

```
let count = ticker(Duration::from_millis(10))
    .count();
count.run(RunMode::RealTime, RunFor::Cycles(100))?;
```

now — the engine owns everything

```
#[op(build = scale, fluent)]
impl Op for Scale {
    type Cfg    = f64;           // config
    type State  = ();           // engine-owned
    type In<'a> = (&'a f64,);    // passed in
    type Out    = f64;
    const ACTIVATION: Activation = Activation::NONE;

    fn cycle(cfg: &mut f64, _state: &mut (),
        input: (&f64,), _ctx: &mut Ctx<'_>)
        -> Result<Tick<f64>>
    {
        Ok(Tick::Value(input.0 * *cfg))
    }
}
```

owns: nothing — cfg, state and inputs all arrive as arguments

```
let g = GraphBuilder::new();
let count = g.ticker(Duration::from_millis(10))
    .count();
g.build().run(RunMode::RealTime, RunFor::Cycles(100))?;
```

WHAT IT BUYS

Two engines, one copy of the semantics

Engine	Who owns state	Dispatch
Interpreted	the builder's slots	one dyn call per node
Compiled	locals in a generated fn	monomorphized, inlinable

One implementation — which is why they cannot drift.

...and you choose the boundary, not the engine

A **nested island** mounts the compiled emission as *one node* of an interpreted graph. Not a third engine — which is why it lands *between* the two.

You write the wiring once, in plain Rust

```
wingfoil::nitro! {  
    fn odds_evens(g: &GraphBuilder) -> Stream<String> {  
        let count      = g.ticker(PERIOD).count();  
        let is_even     = count.map(|i| i.is_multiple_of(2));  
        let is_odd      = is_even.map(|b| !b);  
        let odd_str     = count.filter(&is_odd).map(|i| format!("{i} is odd"));  
        let even_str    = count.filter(&is_even).map(|i| format!("{i} is even"));  
        let labelled    = odd_str.merge(&even_str);  
        labelled  
    }  
}
```

Out comes `odds_evens::interpreted()`, `odds_evens::compiled()` and `odds_evens::nested()` — from the same tokens.

The wiring function stays **valid, reviewable Rust** either way. That property was deliberately never traded away: `count` is read three times, so it is a shared apex node — the macro derives a real DAG, not a chain.

...AND THIS IS WHAT IT BECOMES

One function. One local per node. No dyn.

```
let mut __k = Kernel::new(run_mode, run_for);

let mut __cfg_wf_anon_1    = PERIOD;
let mut __state_wf_anon_1 = __wf_op_ticker_seed_state(&__cfg_wf_anon_1);
let mut __v_wf_anon_1     = __wf_op_ticker_seed_value(&__cfg_wf_anon_1);
// ... one cfg / state / value local per node, 11 nodes ...

let mut __dirty = [false; 11usize];
while __k.begin_cycle(&mut __dirty) {
    let __t_wf_anon_1 = /* activation */ && {
        let mut __ctx = Ctx::new(&mut __k, 0usize);
        match __wf_op_ticker_cycle(&mut __cfg_wf_anon_1,
                                   &mut __state_wf_anon_1, (), &mut __ctx)
            .context("node 0 (wf_anon_1: ticker) cycle")? {
            Tick::Value(__val) => { __v_wf_anon_1 = __val; true }
            Tick::Silent(__val) => { __v_wf_anon_1 = __val; false }
            Tick::Quiet       => false,
        }
    };
};
// ... 10 more, emitted in topological order ...
}
```

abridged · elisions marked · all 1,730 lines committed verbatim beside the example:
github.com/wingfoil-io/wingfoil → [examples/core/dual_mode/expanded/](https://github.com/wingfoil-io/wingfoil/blob/main/examples/core/dual_mode/expanded/)

BOTH ENGINES ARE IN THE BINARY

Flip tiers without a rebuild

```
$ WINGFOIL_TIER=interpreted dual_mode
0.000_000 odds/evens "1 is odd"
0.010_000 odds/evens "2 is even"
0.020_000 odds/evens "3 is odd"
...
ran 10 cycles on the interpreted tier
```

```
$ WINGFOIL_TIER=compiled dual_mode
0.000_000 odds/evens "1 is odd"
0.010_000 odds/evens "2 is even"
0.020_000 odds/evens "3 is odd"
...
ran 10 cycles on the compiled tier
```

Same binary. Same output. Different engine.

Develop interpreted — it honours the tracing span features and supports dynamic graph surgery, neither of which the monomorphized tiers have. **Deploy compiled.** `Tier::default()` falls back to the build profile, so the usual workflow needs no call-site change at all.

RESULTS

The numbers

~0.3 ns

per node cycle, compiled
~12 ns interpreted

4.4–37×

compiled vs interpreted

0.56–0.84×

new interpreted vs the *old* engine

Workload	Nodes	legacy	interpreted	compiled	nested
dense_chain	37	7.95 ms	6.64 ms	187 μs	931 μs
fanout	103	17.05 ms	11.99 ms	324 μs	1.43 ms
fan_in_256	260	38.08 ms	31.60 ms	2.55 ms	3.10 ms
sparse_wide	781	3.07 ms	2.01 ms	355 μs	896 μs

Shared 4-core VM — read the ratios, not the times. Eight workloads; four shown.

EXTENSIBILITY

Your op is not a second-class citizen

```
#[op(build = my_thing)]           // on your Op impl, in your crate  
impl Op for MyThing { ... }      // → the interpreted builder method  
                                // → the nitro! forwarders compiled emission uses
```

Within **2.4%** of a built-in on the compiled path — no edit to the macro crate, no table to register in.

PART THREE

What's next

Down the stack, up the stack,
and where we could use help.

DOWN THE STACK

Closer to the metal

- 1 **Core pin** — the graph thread on an isolated core.
- 2 **Kernel bypass** — Onload, then ef_vi/DPDK. This is the one that buys the wire-to-trade number we currently decline to claim.
- 3 **Zero-allocation I/O** — no allocator on the hot path.
- 4 **Verilog / FPGA** — the graph as gateware, with the backtest as its testbench.

Same graph definition. Further down.

UP THE STACK

A platform, not just an engine

The missing layer

Simulated exchange · OMS / EMS · position
keeping · risk · trade booking · venue adapters

None of it is a new kind of thing

Position keeping is a *fold*. Risk is a *filter*. PnL is a *join*.
The simulated venue is just an *op*.

HELP WANTED

We'd love contributors

- Around **20 people** so far.
- **Good first issues** that really are — each names the file to change and code to copy.
- The bigger projects are separable, and each links to a design rather than a blank page.

Also genuinely useful

Where the docs lost you. The benchmark you don't believe. A project wingfoil would suit — or plainly wouldn't.

THANK YOU



If you work in financial services — or want to build a system to trade your own account — **please do reach out.**

github.com/wingfoil-io/wingfoil · Apache-2.0

```
cargo add wingfoil · pip install wingfoil
```

hello@wingfoil.io · [Discord](#) · [GitHub Discussions](#)

